

STRUKTUR DATA

Pertemuan 6

Queue (Antrian)

Definisi

Antrian merupakan suatu struktur data linear. Konsepnya sama dengan Tumpukan, perbedaannya adalah operasi penambahan dan penghapusan pada ujung yang berbeda. Penghapusan dilakukan pada bagian **DEPAN (FRONT)** dan penambahan berlaku pada bagian **BELAKANG (REAR)**. Elemen-elemen di dalam antrian dapat bertipe data **integer, real, rekord** dalam bentuk **sederhana** atau **terstruktur**.

Sistem Pengaksesan

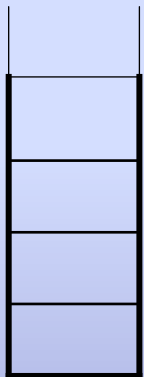
Antrian disebut juga "*Waiting Line*" yaitu penambahan elemen baru dilakukan pada bagian **BELAKANG** dan penghapusan elemen dilakukan pada bagian **DEPAN**. Sistem pengaksesan pada Antrian menggunakan sistem FIFO (*First In First Out*), artinya elemen yang pertama masuk itu yang akan pertama dikeluarkan dari Antrian. Implementasi Antrian dapat ditemukan antara lain :

1. Penjualan karcis kereta, Bioskop
2. Penjadualan pencetakan (Spooling System), misal Print Manager.
3. Penjadualan pemakaian CPU, pada Client-Server
4. Pemakaian jalur I/O (Input/Output), pada sistem computer
5. Penyimpanan barang di Apotek.

Contoh Antrian

Contoh Antrian Kosong, Antrian dengan 1 elemen dan Antrian dengan N elemen.

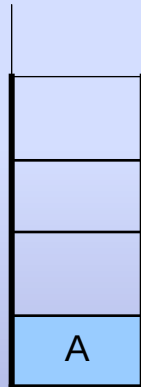
Antrian
kosong



Belakang = 0

Depan = 0

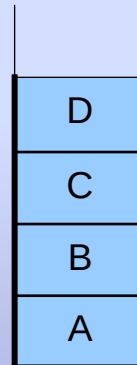
Antrian
1 Elemen



Belakang = 1

Depan = 1

Antrian
N Elemen



Belakang = 4

Depan = 1

Kamus Data

Berikut ini pendeklarasian struktur Antrian dalam kamus data :

Kamus Data

Const

MAKSQ = 8; {Kapasitas maksimal dari Antrian, misalnya 80 elemen}

Type

Jenis Elemen = char;

Antrian = record

Item : Array[1..MAKSQ] of jenisElemen;

Depan : 0..MAKSQ;

Belakang : 0..MAKSQ;

Antrian adalah struktur data bertipe record yang terdiri dari field

1. Elemen, bertipe larik/array dengan indek dari 1 sampai dengan MaksQ.
2. Depan, bertipe integer berkisar dari 0 (saat kosong) sampai dengan MaksQ
3. Belakang, bertipe integer berkisar dari 0 sampai dengan MaksQ.

Operasi Dasar pd Antrian

1. `CreateQueue(Q)` : Membuat Antrian baru Q, dengan jumlah elemen kosong.
2. `MakeNullQ (Q)` : Mengosongkan Antrian Q, jika ada elemen maka semua elemen terhapus.
3. `EmptyQ` : Antrian kosong? Menguji apakah Antrian Q kosong.
4. `FullQ` : Antrian penuh? Menguji apakah Antrian Q penuh
5. `TambahkanQ/ Insert (x,Q)` : memasukkan elemen baru x ke dalam Antrian Q
6. `AmbilQ/Remove (Q,x)` : mengeluarkan elemen depan pada Antrian Q.

Ilustrasi operasi Tambah/InsertQ dan Hapus/ RemoveQ terhadap Antrian

NO.	OPERASI	ISI ANTRIAN	DEPAN	BELAKANG
1.	CREATEQ(Q)	Kosong	0	0
2.	Tambah/InsertQ('a',Q)	a	1	1
3.	Tambah/InsertQ('b',Q)	a b	1	2
4.	Tambah/InsertQ('c',Q)	a b c	1	3
5.	Ambil/RemoveQ(Q,x)	b c	2	3
6.	Tambah/InsertQ('d',Q)	b c d	2	4
7.	Tambah/InsertQ('e',Q)	b c d e	2	5
8.	Ambil/RemoveQ(Q,x)	c d e	3	5
9.	Ambil/RemoveQ(Q,x)	d e	4	5
10.	Ambil/RemoveQ(Q,x)	e	5	5

Underflow & Overflow

Apa yang terjadi bila dilakukan Ambil/RemoveQ(Q, x) sebanyak dua kali lagi ? Underflow, artinya antrian kosong tidak ada elemen yang dapat diambil. Apa yang terjadi bila dilakukan Tambah/InsertQ(x,Q) sebanyak sepuluh kali, jika kapasitas Antrian adalah 5 lagi ? Overflow, artinya antrian penuh tidak ada elemen yang dapat dimasukkan ke dalam Antrian.

Algoritma InsertQ

Algoritma : Tambah/InsertQ Antrian

1. [Periksa Antrian, apakah penuh]
Jika FullQ(Q) maka cetak OVERFLOW
Return
2. [Naikkan nilai Belakang]
Jika EmptyQ(Q) maka {Antrian kosong}
DEPAN = 1 dan BELAKANG = 1
Jika BELAKANG = N maka {Antrian dapat diisi}
BELAKANG = BELAKANG + 1
3. [Masukkan elemen baru]
Antrian [BELAKANG] = ELEMEN
4. [Jika Belakang = MaksQ dan Depan $\neq$ 1, lakukan penggeseran]
Jika BELAKANG = MaksQ AND DEPAN $\neq$ 1 maka
GESERAntrian(Q)
5. Return

Algoritma RemoveQ

Algoritma : Ambil/RemoveQ Antrian

1. [Periksa Antrian, apakah kosong]
Jika EmptyQ(Q) maka cetak UNDERFLOW
Return
2. [Ambil/Remove elemen di posisi Depan]
Elemen = Antrian[DEPAN]
3. [Naikkan nilai Depan]
Jika DEPAN = BELAKANG maka {Antrian ada 1 elemen}
DEPAN = 0 dan BELAKANG = 0
Jika tidak DEPAN = DEPAN + 1
4. Return

Contoh Soal

Q = [O,S,A,M,A]

Lakukan operasi Queue berikut :

- | | |
|--------------------|----------------------|
| 1. Insert [Q,A] | 6. Insert [Q,U] |
| 2. Remove [Q,item] | 7. Insert [Q,O] |
| 3. Remove [Q,item] | 8. Insert [Q,E] |
| 4. Insert [Q,W] | 9. Remove [Q,item] |
| 5. Remove [q,item] | 10. Remove [Q, item] |

Jawab :

1. Insert [Q,A]

Overflow karena kelebihan data

2. Remove [Q,item]

3. Remove [Q,item]

Q = [__,S,A,M,A]

Q = [__,__,A,M,A]

Front (Q) = S

Front (Q) = A

Rear (Q) = A

Rear (Q) = A

Noel (Q) = 4

Noel (Q) = 3

Maxs (Q) = 5

Maxs (Q) = 5

Isempty (Q) = false

Isempty (Q) = false

Penyajian Antrian... 1

Antrian dapat disajikan dari dalam komputer dalam berbagai cara. Biasanya dengan menggunakan One -Way -List (Linier Linked List) ataupun menggunakan array. Kalau tidak disebutkan lain, maka Antrian kita sajikan dalam array Queue, dengan dilengkapi 2 variabel penunjuk. FRONT berisi lokasi dari elemen depan antrian dan REAR berisi lokasi dari elemen belakang antrian. Nilai front = null menunjukkan bahwa Antrian adalah hampa.

Gambar berikut : menunjukkan bagaimana menyajikan suatu antrian dalam sebuah array queue dengan N elemen dan menunjukkan bagaimana melakukan pemasukan dan penghapusan elemen antrian.

Queue

Front : 1

Rear : 4

AAA	BBB	CCC	DDD					
①	2	3	④	5	6	7		N

a

Penyajian Antrian...2

Queue

Front : 2

Rear : 4

	BBB	CCC	DDD					
1	2	3	4	5	6	7		N

b
Queue

Front : 2

Rear : 6

	BBB	CCC	DDD	EEE	FFF			
1	2	3	4	5	6	7		N

c

Penyajian Antrian...3

Queue

Front : 3

Rear : 6

		CCC	DDD	EEE	FFF			
1	2	3	4	5	6	7		N

d

Dapat kita lihat bahwa setiap kali **penghapusan**, nilai lokasi **front** akan **ber + 1** untuk setiap kali **pemasukan** elemen, nilai **rear** akan **ber + 1** hal ini berakibat bahwa setelah pemasukan elemen ke-N (berawal dari antrian hampa). Maka lokasi Queue (N) telah diduduki, disini mungkin saja tidak sebanyak N elemen ada dalam antrian (karena sudah dilakukan penghapusan).

Penyajian Antrian...4

Untuk melakukan pemasukan berikutnya, yaitu memasukkan elemen item, kita dapat menggunakan lokasi Queue (1) datang sesudah Queue (n) di array dalam berdasarkan asumsi ini, maka rear adalah 1. Gambar berikut ini memperlihatkan antrian yang disimpan dalam array dengan lokasi memori sebagai array sirkular.

Queue Circular...1

a) Pada awal hampa

Queue

Front : 0

Rear : 0

1	2	3	4	5

b) A dan B dimasukkan

Queue

Front : 1

Rear : 2

A	B			
1	2	3	4	5

c) C,D dan E dimasukkan

Queue

Front : 1

Rear : 5

A	B	C	D	E
1	2	3	4	5

Queue Circular...2

d) A,B,C dihapus

Queue

Front : 4				D	E
Rear : 5	1	2	3	4	5

e) F dimasukkan

Queue

Front : 4	F			D	E
Rear : 1	1	2	3	4	5

f) D dihapuskan

Queue

Front : 5	F				E
Rear : 1	1	2	3	4	5

Queue Circular...3

g) G dan H dimasukkan

Queue

Front : 5

Rear : 3

F	G	H		E
1	2	3	4	5

h) E dihapuskan

Queue

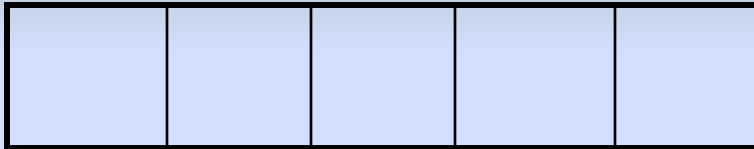
Front : 1

Rear : 3

F	G	H		
1	2	3	4	5

Soal

1. Array Queue = Null dengan max (Q) = 5



- a. Insesrt KLM
- b. Insert AB
- c. Remove 2 elemen
- d. Insert FG
- e. Remove item
- f. Remove item
- g. Insert H
- h. Remove item

Berapakah F,R,I ?

2. Diketahui Data : P,Q,R,S,T,U,V dimasukkan ke dalam stack kosong, kemudian di POP sebanyak 4 elemen dan langsung dimasukkan ke dalam suatu Queue. Setelah itu di remove di Queue 1 elemen dan langsung dimasukkan lagi ke dalam stack awal.

- a. Tentukan isi stack terakhir
- b. Tentukan F,R,I dari Queue